

COPILOTTRAINING
TECHNICAL BRIEFING

Harness Fundamentals

Getting Inside the Agent Loop

Understand the loop, then bring AI into your existing apps with tools, controls, and telemetry.

Extend and Embed · CopilotTraining Tech Talk · 60 min · @github/copilot-sdk 1.0.26

🤔 THE CORE QUESTION

"How do I get inside the agent loop and change how it behaves?"

A harness is just a loop: ask the model, run what it asks for, feed back the result, repeat.

Take over the right job, and the agent sees, uses, and does what your workflow needs.

Developers

Already use Copilot in the editor or CLI and want the agent inside their own tools

Platform engineers

Decide which rules run in every harness and which belong in one app

Architects

Choose which existing app methods an agent may request and what access it needs

~170 lines

pi's entire agent loop, one function you can read on one screen

5 places to step in

Loop, context, tools, control, evidence: every hook or callback fills one

Same loop in Copilot

The SDK starts the Copilot CLI runtime; your code steps in at the same five places

📖 TABLE OF CONTENTS

PART 1

What a Harness Is

Five jobs, read in pi's source

Loop, context, tools, control, and evidence, each on one slide of real code

PART 2

Drive GitHub's Harness

Bring AI into the apps you already own

Call existing methods through tools, enforce app permissions, and stream progress

PART 3

Decide What It Sees

One policy, a blind tester, and its evidence

Hide the implementation and let a referee name what the tests found

PART 4

Own It

Surrounding controls, pre-flight, named owner

Share a harness others can run, and know who reads each report

PART 1

What a Harness Is

Read pi as source, one slide per job: loop, context, tools, control, and evidence.

The loop

One function of about 170 lines runs every turn

Tools + control

find hands off to fd; beforeToolCall says no

Evidence

Every step is an event any subscriber can read

```
Code excerpts: pi · earendil-works/pi@f9daca3 · MIT License · trimmed, read as source
↳ 5 jobs: loop · context · tools · control · evidence
```

A harness is a loop with five places to step in

Each job is a point in the loop: the question it answers, and where pi does it

Loop	When does the agent take another turn, and when does it stop?	`runLoop`
Context	What does the model see on this turn?	`transformContext`
Tools	What can the agent do, and how does each action run?	`write · find`
Control	Which calls actually run?	`beforeToolCall`
Evidence	What happened, in a form someone can inspect?	`emit · afterToolCall`

pi is one small open-source harness among many; the five jobs are the part that transfers

That's the whole harness: pi's loop is one function of ~170 lines

pi · packages/agent/src/agent-loop.ts#L188-L337 (trimmed)

```
while (true) {                                                    // LOOP: outer, for follow-ups
  while (hasMoreToolCalls || pendingMessages.length > 0) {        // LOOP: one turn per pass
    // … queued steering messages join currentContext.messages    // CONTEXT
    const message = await streamAssistantResponse(currentContext, config, signal, emit, streamFunction);
    const toolCalls = message.content.filter((c) => c.type === "toolCall");
    if (toolCalls.length > 0) {                                   // TOOLS + CONTROL
      const executedToolBatch =
        message.stopReason === "length"
          ? await failToolCallsFromTruncatedMessage(toolCalls, emit)
          : await executeToolCalls(currentContext, message, config, signal, emit);
    }
    await emit({ type: "turn_end", message, toolResults });       // EVIDENCE
    pendingMessages = (await config.getSteeringMessages?.()) || [];
  }   // … then follow-up messages restart the outer loop, or it breaks
}
```

🔧 Tool calls keep it running

No tool calls ends the turn loop; results go back into context.

🧭 People steer mid-run

Steering joins before the next turn; follow-ups restart it.

📡 Every turn emits an event

Where a UI, log, or budget attaches.

pi rebuilds what the model sees before every model call

pi · packages/agent/src/agent-loop.ts#L398-L404

```
let messages = context.messages;
if (config.transformContext) {
  messages = await config.transformContext(messages, signal);
}
const llmMessages = await config.convertToLlm(messages);
```

✂️ transformContext

Prunes, summarizes, or injects messages before each model call.

🚀 prepareRequest

Runs just before the call and can swap the model or thinking level for the next turn.

📣 declareToolChanges

Tells the model when its tool list changed.

💡 The harness decides the model's view on every turn, so code that controls context shapes the agent's whole view.

A harness can bring its own tools: pi's 'write'

pi · packages/coding-agent/src/core/tools/write.ts (trimmed)

```
const writeSchema = Type.Object({
  path: Type.String({ description: "Path to the file to write (relative or absolute)" }),
  content: Type.String({ description: "Content to write to the file" }),
});
export const writeToolSystemPromptContribution = {
  snippet: "Create or overwrite files",
  guidelines: ["Use write only for new files or complete rewrites."],
} as const;
return withFileMutationQueue(absolutePath, async () => {   // … inside execute(), later in the file
  throwIfAborted();
  await ops.writeFile(absolutePath, content);              // … after ops.mkdir; then returns success
```

🧩 A tool is schema + code

The model fills the schema; the harness runs the code.

🏗️ Built in, no shell

write calls no OS command; pi wrote every line.

🔀 Your rules inside

Queue, abort, and ops: disk, SSH, or anywhere.

🚪 Build it or borrow it: pi's 'find' borrows the fd binary by default; the harness still sees call and result.

Before any tool runs, the loop asks your code: run it or block it?

pi · packages/agent/src/agent-loop.ts#L738-L765 (trimmed)

```
if (config.beforeToolCall) {
  const beforeResult = await config.beforeToolCall(
    { assistantMessage, toolCall, args: validatedArgs, context: currentContext }, signal);
  if (beforeResult?.block) {
    const result = createErrorToolResult(beforeResult.reason || "Tool execution was blocked");
    return { kind: "immediate", result, isError: true };
  }
}
// pi · coding-agent/src/core/agent-session.ts#L652-L682: the product installs its policy
this.agent.beforeToolCall = (context) => this._beforeToolCall(context);
this.agent.afterToolCall = (context) => this._afterToolCall(context);
```

↩ Blocked? The agent hears why

Your reason goes back to the model; it tries another way.

🔭 Your code sees everything

The tool, its arguments, and the conversation so far.

🎀 Half-written calls never run

If a reply hits the length limit, its tool calls fail.

🧩 pi's own coding agent plugs its add-ons in right here; the loop itself never changes.

pi reports every step as an event any subscriber can read

pi · packages/agent/src/agent-loop.ts#L880-L937 (trimmed)

```
if (config.afterToolCall) {
  const afterResult = await config.afterToolCall(
    { toolCall: prepared.toolCall, args: prepared.args, result, isError, /* … */ }, signal);
  if (afterResult) { result = { ...result, content: afterResult.content ?? result.content /* … */ }; }
}
await emit({ type: "tool_execution_end", toolCallId: finalized.toolCall.id,
  toolName: finalized.toolCall.name, result: finalized.result, isError: finalized.isError /* … */ });
// agent.ts#L275-L278
subscribe(listener: (event: AgentEvent, signal: AbortSignal) => Promise<void> | void): () => void {
  this.listeners.add(listener);
  return () => this.listeners.delete(listener);
}
```

afterToolCall can rewrite
Redact or trim before the model reads it.

Audit-ready events
Tool, call, result, error, and duration.

Anything can listen
subscribe feeds a UI, a log, or a budget.

agent_start · turn_start · message_start/update/end · tool_execution_start/update/end · turn_end · agent_end

PART 2

Drive GitHub's Harness

Put the loop inside an app you already own, with its methods, permissions, and UI.

Your app is the host

Host = the app running the SDK; it can be one you already use

Your methods become tools

The model requests a named tool; your handler calls app code

Your rules, your telemetry

App permissions stay in force; events feed your UI and logs

```
Integration sketch · existing release dashboard + @github/copilot-sdk
↳ What blocks this release? → app method → readiness result → dashboard
```

Same five jobs, different code: the SDK has its own names for each

Copilot SDK names in the middle; the pi code you just saw, for comparison, on the right

Loop	The runtime; your app drives createSession, sendAndWait, abort	`pi: runLoop`
Context	systemMessage (append, customize, replace), the prompt, model	`pi: transformContext`
Tools	defineTool wraps app methods; availableTools names what exists	`pi: write · find`
Control	onPermissionRequest, hooks.onPreToolUse, repository hooks	`pi: beforeToolCall`
Evidence	session.on feeds your UI and telemetry; services record results	`pi: emit · afterToolCall`

Documented: the SDK talks to the Copilot runtime over JSON-RPC and starts and stops it for you

A dashboard action starts the loop and displays its answer

release-assistant.ts (integration sketch)

```
const client = new CopilotClient();
const session = await client.createSession({
  model: "auto", tools: [readinessTool],
  availableTools: ["release_readiness"],
  onPermissionRequest: releasePermissions,
});
const reply = await session.sendAndWait({
  prompt: `What blocks release ${selectedRelease.id}?`,
});
ui.showAnswer(reply?.data.content);
```

🖥️ Your app starts it

A UI action supplies the selected release and the question.

⏳ sendAndWait

The runtime handles model turns and tool calls; your app gets the reply.

🖼️ Your app shows the answer

ui.showAnswer delivers the reply inside the dashboard people already use.

🔁 Same loop as pi. The SDK hands you the five places to step in.

Give the model a clear job inside your app

release-assistant.ts · session options (integration sketch)

```
systemMessage: {
  mode: "customize",
  sections: { custom_instructions: { action: "append", content:
    "You assist a release engineer inside the release dashboard. " +
    "Use release_readiness for current status; name each blocker. " +
    "If the tool cannot answer, report the gap. " +
    "Explain the next step for a person to approve." } },
}
```

systemMessage

GitHub's built-in instructions to the model, in named sections.

mode: customize

Edit one section; every other section stays as GitHub wrote it.

custom_instructions

Names the app task, the source of facts, and what to do when facts are missing.

Instructions steer the model; the next two slides enforce limits in code.

A custom tool can call a method your app already has

release-tools.ts (integration sketch; releaseService is app code)

```
const readinessTool = defineTool("release_readiness", {
  description: "Read the current blockers for a release.",
  parameters: { type: "object", properties: {
    releaseId: { type: "string" } }, required: ["releaseId"], additionalProperties: false },
  handler: async ({ releaseId }: { releaseId: string }) ⇒
    releaseService.getReadiness(user, releaseId),
});
// Session options:
tools: [readinessTool],
availableTools: ["release_readiness"]
```

🔗 handler → app method

The SDK calls your handler; it calls releaseService.getReadiness.

🛂 Keep app authorization

The service checks the authenticated user and the requested release.

✅ Expose just this tool

tools adds it; availableTools hides shell, files, and other tools.

💡 The model chooses when to request a tool; your app chooses which method it calls and what data comes back.

Your app controls tool requests and checks user access

release-permissions.ts (integration sketch)

```
const releasePermissions: PermissionHandler = (request) ⇒
  request.kind ═══ "custom-tool" && !request.managedApprovalRequired &&
  request.toolName ═══ "release_readiness"
    ? { kind: "approve-once" }
    : { kind: "reject", feedback: "Readiness only. Ask a release owner." };
// Session options:
onPermissionRequest: releasePermissions,
enableFileHooks: true
```

Allow one operation

The permission handler approves only release_readiness.

Authorize inside the service

Tool approval never grants access to every user or release.

Reject with a next step

Other requests get a reason and a route to the release owner.

Load repository hooks

enableFileHooks loads repo rules; a repo deny still wins.

SDK approvals gate tool requests; your service checks user access and any required human approval.

Feed progress into your app UI and tool events into its telemetry

release-events.ts (integration sketch; ui and telemetry are app code)

```
// Set streaming: true when creating the session; subscribe before sending.
session.on("assistant.message_delta", (e) => ui.appendText(e.data.deltaContent));
session.on("tool.execution_start", (e) =>
  ui.showTool(e.data.toolName));
session.on("tool.execution_complete", (e) =>
  telemetry.record("tool.complete", {
    sessionId: session.sessionId, toolCallId: e.data.toolCallId,
    success: e.data.success,
  }));
session.on("session.idle", () => ui.finishResponse());
```

Progress in your UI

Stream the answer and show which tool the agent requested.

Events in your telemetry

Record the session, tool call, and whether execution succeeded.

Business audit stays in the app

The service records access decisions and the actual release result.

Tool execution success means the call completed; ready: false can still be its business result.

AI brings your app's existing methods into a new workflow

🧭 One question becomes an app workflow

Engineer asks: What blocks v2.4?
The dashboard supplies the selected release

Agent requests release_readiness
Your handler calls releaseService.getReadiness

Service returns ready: false
Example blocker: integration tests failed

Dashboard shows answer and progress
The engineer sees the blocker and chooses the next action

🎛️ Your app keeps the controls

One named read-only operation
No shell, file access, or publish tool exposed

Service checks the app user
A supplied release ID never grants access

SDK events plus the service audit
Trace the request and verify the actual result

Missing facts or failed access?
Show the error; a release owner resolves it

💡 Integration sketch: AI uses your app's existing capabilities through methods you deliberately expose.

PART 3

Decide What It Sees

Same host, different view: one visibility policy produces a blind test suite and a referee's verdict.



Two checkpoints

onPreToolUse and onPermissionRequest share a policy



Blind test writer

Contract only, implementation hidden, host reruns



Evidence

JSONL from three layers, joined by sessionId

```
host rerun · TAP output · read-only referee session with BUGS.md
↳ 10 passing · 2 failing · 1 undocumented mismatch
```

One visibility policy feeds both SDK checkpoints

examples/host-kit/visibility.ts · blind-tests app

```
const policy = createVisibilityPolicy({
  repoRoot: checkout.dir,
  hidden: isHiddenFromBlindTester,       // never visible, any access kind
  readable: isBlindReadable,             // optional allowlist for reads
  writable: isBlindWritable,             // where the agent may create files
  shellCommands: BLIND_SHELL,            // executables it may run
  shellTextDeny: BLIND_TEXT_DENY,        // text that reaches hidden material indirectly
  customTools: new Set(["api_contract"]),
  hiddenFeedback: "You are testing a black box.  …  Use api_contract and live HTTP calls.",
});
createSession({ …, onPermissionRequest: policy.onPermissionRequest,
  hooks: { onPreToolUse: policy.onPreToolUse } });
```

onPreToolUse

Raw arguments: catches a grep sweeping a hidden folder.

onPermissionRequest

Parsed request: hidden reads, stray writes, shell commands.

Beyond a hook (Tested)

Write-content checks; per-segment command allowlist.

🔭 Like pi's beforeToolCall, the host holds session state across events.

A blind tester surfaced an API contract mismatch nobody had documented

Does the API behave the way its contract says, judged by someone who never saw the code?

10 / 2

test status: passing / failing on the host's own rerun

Observed 2026-10-26 · TAP output · 18 tool calls, 19 allowed, 2 denied

📄 **Sees only api_contract**

SETUP.md endpoints, bug hints stripped; live HTTP calls.

🙈 **Hidden: source, docs, BUGS.md**

Writes only __tests__/blind/; host serves on 5299.

🐞 **Finding: POST /api/characters → 500**

Referee: BUGS.md #19, Missing Input Validation.

❓ **Finding: seasonId filter ignored**

Referee: undocumented. Docs say seasonId; route reads season_id.

⚖️ A read-only referee labels each failure; the API owner decides which side changes.

Three things no harness can see

Inside a process a tool starts

The harness approves running npm test; what that code does next belongs to the OS.

Scripts it runs have full process access

For a hard wall, run the agent in a container

Anything that isn't a tool call

The harness only decides what the agent asks its tools to do.

Other processes, users, and the network

OS accounts and network rules cover those

Whether the result is right

The harness records what happened; judging it is a separate job.

Tests, reviews, and evidence answer that

A named person decides what ships

Same as pi's find → fd: the harness sees the call and its result; the OS runs the rest.

Every run leaves one log, with every layer joined by sessionId

.runs/my-app/events.jsonl (example)


```
{"layer":"repo-hook","event":"hook.end","sessionId":"909bf25c-…","hook":"preToolUse","success":true}
{"layer":"permission-handler","sessionId":"909bf25c-…","path":"secrets/.env","decision":"deny"}
{"layer":"runtime","event":"tool.complete","sessionId":"909bf25c-…","tool":"view","success":true}
{"layer":"app","event":"session-end","sessionId":"909bf25c-…","toolCalls":18,"allow":16,"deny":2}
```


🔗 One sessionId joins every layer

Repo hooks, your permission handler, and the runtime, lined up for one session.

⚖️ Decisions apart from actions

What was allowed or denied, and separately, what actually ran.

🏷️ Names, rules, and counts

Tool, path, and rule; file contents and command output stay out.

📌 Build reports from the log of what the tools did; the agent's own summary is a claim to check.

PART 4

Own It

Know the controls other people already own, then share your harness with a pre-flight and a named owner.



Surrounding controls

Policy hooks, models, billing, tools, process



Pre-flight

Pins, clean checkouts, bounds, tested rules



Named owner

Every report goes to someone who decides

```
A harness you share becomes a harness someone owns
↳ Which of the five jobs do you take over?
```

Your harness runs inside controls other people already own

What each owner decides, and what your app verifies before you share it

Policy hooks	Load machine-wide before repo hooks; run under disableAllHooks	Platform team
Access	Who uses Copilot, on which surfaces; check your app identity	Org admins
Models	Admins set availability; model auto or a named model is a request	Admins + app
Billing	User or token route follows Copilot policy; BYOK follows provider	Billing admins
Tools + MCP	managedApprovalRequired requests need a human approval path	Admins + app
Process	OS account, container, network: the real boundary for installs	Platform owners

Documented in the hooks reference and Copilot policies; verify each against the identity your app runs as

Before sharing, run a pre-flight and name an owner

✅ Pre-flight before someone else runs it

Pin the SDK; record the runtime version
Every report names the SDK and runtime versions

Start from a clean, pinned checkout
Tool budget, timeout, cleanup in finally

Unit-test every visibility rule
Against real request shapes; set enableFileHooks: true

Confirm auth route, models, billing
For the people who will run it

🧭 How deep to go

A note at session start steers it
→ sessionStart hook

A call must never run, in any harness
→ preToolUse repo or machine policy hook, kept fast

Needs persona, tools, visibility, budget, report
→ SDK host with enableFileHooks: true

Name the owner of every report
Docs owner: friction; API owner: blind failures

🎯 Your move: which of the five jobs do you take over for your workflow, and who reads the report?

THE VALUE RECAP

From Watching the Loop to Hosting It in Your App

Before

- The agent is a black box you prompt from outside
- Your app already has useful methods behind its buttons
- Rules live in prompts and reviewer memory
- The agent's own summary stands in for the result

After

- Just a loop: you choose where to step in, five known places
- Expose selected app methods as named tools
- App authorization, permission handlers, and repo hooks
- Stream progress to your UI; verify results in service logs

~170
lines in pi's whole runLoop

5 jobs
places your app can step into the loop

1 method
a starting scope for an embedded assistant

What could AI unlock inside the tools your team already uses?

Wrap selected app methods as tools; keep permissions and telemetry in the app

CI console	Calls rerunTest and compareRuns to investigate a flaky build	`Tools + Evidence`
Upgrade crew	Your dependency portal tests upgrade branches in isolated workers	`Loop + Control`
Release UI	Calls getReadiness and draftRollout; your app gates deployment	`Tools + Control`
Runbook drill	Follows an incident runbook in a sandbox; flags each unclear step	`Context + Evidence`
Exam grader	Your learning app grades submitted tests; the answer key stays hidden	`Control`
Issue→release	Your dev portal coordinates stages; app approval gates each handoff	`All five`

The tag names the job you write code for; start with that one

✅ WHAT YOU CAN DO TODAY

Today

- Watch your agent work and name the job behind each step
- Spot the loop: ask the model, run the tool, feed back the result
- List the rules you rely on that live only in a prompt

This Week

- Read one open-source loop, like pi's runLoop, end to end
- Ask what your agent can see, write, and run, and who decided
- Move one prompt-only rule into a hook and watch it hold

This Month

- Pick an app you own and a method an AI assistant could use
- Sketch one tool wrapper; keep user checks and approvals in the app
- Decide what its UI shows and what its telemetry records

🔑 Key Takeaway

A harness is just a loop: pick the place you want to step in, and start there.

REFERENCES

pi and the Copilot SDK

pi coding agent (MIT) at f9daca3
agent-loop.ts, agent.ts, agent-session.ts, write.ts, find.ts

GitHub Copilot SDK
Architecture, sessions, tools, hooks, examples

@github/copilot-sdk on npm
1.0.26 used in this talk

SDK authentication
Signed-in user, token, and BYOK routes

Hooks and Policy

Copilot hooks reference
Events, preToolUse decisions, policy hook locations

VS Code agent hooks (Preview)
Editor hook format and event coverage

Copilot policies
Enterprise and organization AI controls

Supported AI models
Availability by plan and surface

Related Content

Copilot on GitHub
Pull-request acceptance for agent output

Copilot Primitives
Instructions, prompts, agents, and hooks

Enterprise Patterns



Harness Fundamentals

GETTING INSIDE THE AGENT LOOP

5 jobs

Loop, context, tools, control, evidence: every hook takes over one

Your app

Custom tools call methods you already own

Your controls

Keep permissions, approval gates, UI, and telemetry

Which method in an app you own could become a useful AI tool?